

Return values and local state

Project: Python readiness portfolio

1. Prepare the purpose, prerequisites and materials

Teach the learning cycle

Primary target: Use a returned value in a later calculation

Session · Sequence and minutes · Resources

Session: Session 1 · Understand · 45 min

Sequence and minutes: 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.

Resources: Sections 1–6; record the brief recall in the workbook.

Session: Session 2 · Practise · 45 min

Sequence and minutes: 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

Resources: Sections 7–10; use the matching workbook W2 and misconception check.

Session: Session 3 · Investigate and transfer · 45 min

Sequence and minutes: 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

Resources: Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building. Repeated copies of a case share one recorded attempt; they are not new evidence.

Retrieval answer guidance

Functions and arguments

Without opening the old lesson, explain one key idea from Functions and arguments. Give an example and a mistake that the idea helps you avoid.

Look for: Distinguish a function definition from a function call

Reference example, not the only acceptable wording: Name `show_remaining`; parameters `capacity` and `used`. Arguments 15 and 6 match `capacity = 15` and `used = 6`. Expected printed result 9.

Loops and trace tables

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Look for: Trace variable values before and after each loop iteration

Reference example, not the only acceptable wording: 2, 3 and 4; the stop value is excluded. The body runs three times.

Expressions and operator order

Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.

Look for: Evaluate numeric expressions using Python's grouping rules

Reference example, not the only acceptable wording: $3 * 8 + 2$ or $(3 * 8) + 2$ gives 26. $3 * (8 + 2)$ gives 30 and adds two to each of three packs, not two once.

Use integer inputs, explicit return and simple local assignments. Avoid global variables, mutable defaults and shared objects in this introduction. Python reference: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>. A call has its own local names; return sends a value to the caller. Examples assume valid counts and do not validate them.

Outcomes

Use a returned value in a later calculation

Trace local variables separately for each function call

Detect a missing return and test repeated calls

Preparation

Python 3 editor and `returns.py`

Separate paper spaces for each call and the caller

Expected return value and observed return value table

Prerequisite answer

The call prints 5. The definition alone creates the function without running its body.

End goal for the pictorial case

Trace return values and local state and explain the changed program before running it.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

2. Explain and model the mechanism

Two fictional cupboards have spare spaces. We want to add their spare spaces without copying numbers from the screen. Last week a function printed each answer. This week a function returns its answer so the calling code can store it, combine it or choose to print it.

Assume nonnegative integer counts and used no greater than capacity. This calculation does not validate those assumptions. Work in a Python script so a call does not automatically display its return value as it might in an interactive console.

`return spaces` ends the current function call and sends the value of `spaces` back. The assignment `first = remaining(12, 4)` waits for that value and stores 8. The next call stores 3 in `second`. Neither call prints. The final statement prints 11. A statement after an executed `return` in the same call is not reached.

Term · Meaning

Term: Return value

Meaning: A value a function sends back to its caller

Term: Caller

Meaning: The code that calls a function

Term: Local variable

Meaning: A name assigned inside a function and used in that function call

Term: Scope

Meaning: The region of code in which a name can be looked up

Term: None

Meaning: A Python value representing no result value in this lesson

Two calls return two values

Call · Inside this call · Back in the caller

Call: remaining(12, 4)

Inside this call: spaces becomes 8

Back in the caller: first receives 8

Call: remaining(10, 7)

Inside this call: spaces becomes 3

Back in the caller: second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

first stores 0, second stores 9, and the final print displays 9. Removing only that print leaves the calculations and stored values but no displayed output. Exit: assign answer = remaining(12, 4), then print(answer) and check it is 8 rather than None. Repeat with (10, 7), expecting 3. Passing two tests does not establish all-input correctness or verify the input assumptions.

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a prediction, then explain the deciding step. These teacher notes contain solutions; do not reveal the independent case answer during modelling.

First call before return · 8 · first not assigned yet

First call before return: Second call before return

8: 3

first not assigned yet: first is still 8

First call before return: After both calls

8: Not accessible by this local name

first not assigned yet: first is 8 and second is 3

capacity, used and spaces belong to the current call. The second call gets new parameter values and calculates its own spaces; it does not overwrite first. In a fresh script with no outside variable called spaces, adding `print(spaces)` after the calls raises `NameError`. Use the returned value stored in first or second instead.

Change the calls to `remaining(9, 9)` and `remaining(9, 0)`. What values are stored and what does their sum print? What happens if only the final print statement is removed?

Thinking move · Teacher prompt

Thinking move: Plan

Teacher prompt: What is given? What is the question? Which procedure fits these facts?

Thinking move: Do

Teacher prompt: Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.

Thinking move: Check

Teacher prompt: Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: Return values and local state

Return values survive their calls. A function returns a local calculation that the caller stores.

The facts we will use

Python 3: `def extra(a,b): result=a+b; return result`. Then `first=extra(2,5)`; `second=extra(4,1)`;

```
print(first,second).
```

AI ACADEMY / PYTHON-BRIDGE/9

Return values and local state

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Trace first call

Its local result is 7; return assigns 7 to first.



2. Trace second call

Its new local result is 5; return assigns 5 to second.



3. Read caller values

The final print displays 7 5; the second call does not overwrite first.

CHANGE ONE THING

Add `print(result)` after the calls in a fresh script with no global result.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Trace first call

What happens and why: Its local result is 7; return assigns 7 to first.

Step: 2. Trace second call

What happens and why: Its new local result is 5; return assigns 5 to second.

Step: 3. Read caller values

What happens and why: The final print displays 7 5; the second call does not overwrite first.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Returning passes a value back to the caller. Local names belong to a function call; storing the returned value creates a separate caller binding.

What this example does not establish

The example uses immutable integers; shared mutable objects require additional reasoning about references.

Change one thing in the pictured case

Change: Add `print(result)` after the calls in a fresh script with no global result.

Prediction: `NameError` occurs because that local name is not defined in the caller.

Reason: A returned value and the function's internal name are different things.

Ask while showing the picture

Cover the result. Ask learners to name the inputs and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its evidence. Then change the supplied condition and compare. The picture is a representation; it does not document execution of a real AI model.

3. Demonstrate and guide practice

Time or stage · Teacher action

Time or stage: Session 1 start to 7 minutes

Teacher action: Retrieve the Week 8 distinction between definition and call.

Time or stage: 7 to 20 minutes

Teacher action: Trace the worked example and move returned-value cards to first and second.

Time or stage: 20 to 33 minutes

Teacher action: Complete W1 and trace local names separately for W2.

Time or stage: 33 to 45 minutes

Teacher action: Predict the guided case, run it and explain why assignments do not print.

Time or stage: Session 2 start to 8 minutes

Teacher action: Recall where the caller stores a return value.

Time or stage: 8 to 20 minutes

Teacher action: Run the printing bug, record None and repair it for W3.

Time or stage: 20 to 35 minutes

Teacher action: Collect independent W4 code and predictions before offering hints.

Time or stage: 35 to 45 minutes

Teacher action: Use W5 and the exit to check return control flow and test evidence.

Use the worked example in Sections 3 and 6. Collect a prediction before revealing its result; ask the learner to point to each intermediate fact.

Use separate boxes on paper labelled caller and current call. Move the value card at return, not the local name. Offer the definition header as a hint and record it. Extension: set spaces = 100 outside the correct worked function, call remaining(9, 4), then print the returned result and outside spaces separately. Expect 5 and 100. Explain why the two assignments use different scopes; do not generalize this to mutation of shared objects.

Reduce help deliberately

Completion task: Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

Teacher check: a is 5, b is 0, c is 5. Each call has its own local spaces. That name has not been defined outside the function, so print(spaces) there raises NameError. Print a, b or c to use the saved results.

Start by identifying the given inputs together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood. For partners, alternate explainer and checker; collect a response from every learner before discussion.

Individual reasoning check

Use the worked case for Return values and local state. Proposed explanation: “Printing a value returns that value”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: print displays a value. Without an explicit return, these functions return None.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class. If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial practice question

Trace the two returned values and explain why first stays seven. What happens if the caller tries to print result directly?

Reasoned teacher answer

The returns are 7 and 5, stored independently as first and second. result is local to each call; accessing it outside in this fresh script raises NameError.

Changed-case reasoning: NameError occurs because that local name is not defined in the caller. A returned value and the function’s internal name are different things.

Collect each learner’s explanation before discussion. Use the first missing connection between input, deciding step and result to choose support. This question is worked-case practice, not fresh mastery evidence.

4. Run the investigation and diagnose misconceptions

Printing a value returns that value: print displays a value. Without an explicit return, these functions return None.

A second call overwrites the first saved answer: These integer results are stored separately in the caller. A later call creates new local bindings.

A local name is available anywhere: spaces is assigned inside remaining. In our fresh script it cannot be looked up outside that function.

Zero means the function returned nothing: Zero is a number and supports arithmetic. None is a different value.

Use Section 9 for the evidence ledger. Add a boundary or missing-input case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

Match the support to the first error

What the response shows · Next teaching action

What the response shows: Missing or misread facts

Next teaching action: Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.

What the response shows: Wrong procedure

Next teaching action: Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.

What the response shows: Arithmetic or implementation error

Next teaching action: Trace intermediate values or lines; preserve the first mismatch and retry that step.

What the response shows: Correct result without a reason

Next teaching action: Ask for a trace and a changed case before marking independent understanding.

What the response shows: An unsupported wider claim

Next teaching action: Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal. Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

5. Assess and explain the answers

Guided and exit questions

first stores 0, second stores 9, and the final print displays 9. Removing only that print leaves the calculations and stored values but no displayed output. Exit: assign `answer = remaining(12, 4)`, then `print(answer)` and check it is 8 rather than None. Repeat with (10, 7), expecting 3. Passing two tests does not establish all-input correctness or verify the input assumptions.

W1 Separate returning from displaying

Using the correct worked definition, execute `result = remaining(15, 6)`, then `doubled = result * 2`. State both stored values and what the script displays before a print statement is added.

`result` is 9 and `doubled` is 18. Nothing is displayed by these assignments or the returning function. `print(doubled)` would display 18.

W2 Trace separate calls

Using the correct definition, store `a = remaining(8, 3)`, `b = remaining(8, 8)`, `c = remaining(8, 3)`. State `a`, `b` and `c`. In a fresh script, explain why `print(spaces)` outside the function fails.

`a` is 5, `b` is 0, `c` is 5. Each call has its own local spaces. That name has not been defined outside the function, so `print(spaces)` there raises `NameError`. Print `a`, `b` or `c` to use the saved results.

W3 Repair the returned value

Use the faulty printing version with `answer = remaining(11, 2)`. State the displayed number and the value stored in `answer`. Explain why `print(answer + 1)` fails, then repair the function and predict the repaired output.

The call displays 9 but stores `None`. `None + 1` raises `TypeError`. Replace the internal `print(spaces)` with `return spaces`. The repaired call itself displays nothing; `print(answer + 1)` displays 10. Rerun `(12, 4)` and check that the stored value is 8.

W4 Build a reusable calculation independently

Each tray holds 5 counters. Define `stock(trays, loose)` to return the total using a local variable `total`. Assume nonnegative integer arguments. Store results for `(2, 3)`, `(0, 3)` and `(2, 0)`. Print the sum of the first two stored results. Predict that output and all three stored values. Call `(2, 3)` again to check fresh state.

Define `stock(trays, loose)` with `total = trays * 5 + loose` followed by `return total`. Store values 13, 3 and 10. The sum of the first two is 16. The repeat returns 13. The function should not print; the caller prints 16.

W5 Locate a statement that never runs

A function body has `spaces = capacity - used`, then `return spaces`, then `print("done")`, all at the same indentation. What does `answer = remaining(4, 4)` store and display? Explain whether the final print can run.

The call stores 0 and displays nothing. `return` ends this call, so the following `print("done")` is not reached. Zero is a valid returned value here and is different from `None`.

Independent assessment and fresh retry

W4 must use both arguments, return its local `total`, store 13, 3, 10 and print 16 in the caller. Repeating `(2, 3)` must return 13. After support use fresh retry tiles(packs, singles), with 4 tiles per pack: `(3, 2)`, `(0, 2)`, `(3, 0)` return 14, 2, 12; print the sum of the first two, 16; repeat `(3, 2)` to obtain 14. Record help used and collect predictions before running.

Assess independent transfer

Selected case: Trace both calls and the final expression. If `return total` is removed from the function, does this final print complete? Would a separate `print(total)` outside the function have access to its local variable?

```
def twice(n):
```

```
total = 2 * n

return total

first = twice(3)

second = twice(5)

print(first, second, first + second)
```

Selected case answer

With the return: first=6, second=10, sum16, so output is6 10 16. Removing the return makes both calls return None; evaluating first+second raises TypeError before print runs, so no line is printed. In this shown program total is local, so an outside print(total) raises NameError. Each call has its own local computation.

Dimension · Evidence required

Dimension: Explanation

Evidence required: Names the applicable idea or procedure and ties it to the supplied facts.

Dimension: Trace and result

Evidence required: Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.

Dimension: Limits

Evidence required: States the assumptions, missing evidence or conditions under which the result would change.

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace. Do not penalise an honestly recorded model failure if the learner correctly explains its evidence and limits.

Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case. Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

6. Connect the project, support and next lesson

Add a labelled return values and local state evidence sheet to Python readiness portfolio. Show how you

can use a returned value in a later calculation. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Use separate boxes on paper labelled caller and current call. Move the value card at return, not the local name. Offer the definition header as a hint and record it. Extension: set spaces = 100 outside the correct worked function, call remaining(9, 4), then print the returned result and outside spaces separately. Expect 5 and 100. Explain why the two assignments use different scopes; do not generalize this to mutation of shared objects.

Week 10 uses dictionaries to keep related values together as named fields in a record.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

Plan the delayed return

Suggested return points in this level: python-bridge/10, python-bridge/12. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used. For a new level, revisit the previous capstone before checking the new prerequisites.

Record - Teacher evidence

Record: Learner code and date

Teacher evidence: _____

Record: Concept and case used; prior exposure

Teacher evidence: _____

Record: Explanation / trace / limits

Teacher evidence: _____

Record: Support and first repair

Teacher evidence: _____

Record: Changed case and delayed outcome

Teacher evidence: _____

Record: Next teaching action

Teacher evidence: _____

Use the evidence to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Project evidence from this case

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

7. Prepare, teach and assess

Teaching target: Use a returned value in a later calculation

Prepare the labelled worked-case picture, the matching workbook, fictional input cards and a place for individual responses. Check a local Python 3 environment before class; have a paper-trace version ready.

Session · Plan · Resources

Session: Session 1 · Understand · 45 min

Plan: 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.

Resources: Sections 1–6; record the brief recall in the workbook.

Session: Session 2 · Practise · 45 min

Plan: 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

Resources: Sections 7–10; use the matching workbook W2 and misconception check.

Session: Session 3 · Investigate and transfer · 45 min

Plan: 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

Resources: Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

Before class, solve the supported practice: Using the correct definition, store `a = remaining(8, 3)`, `b = remaining(8, 8)`, `c = remaining(8, 3)`. State `a`, `b` and `c`. In a fresh script, explain why `print(spaces)` outside the function fails.

Reasoned practice answer: `a` is 5, `b` is 0, `c` is 5. Each call has its own local spaces. That name has not been defined outside the function, so `print(spaces)` there raises `NameError`. Print `a`, `b` or `c` to use the saved results.

Check every learner before discussion: Use the worked case for Return values and local state. Proposed explanation: “Printing a value returns that value”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Misconception repair: `print` displays a value. Without an explicit return, these functions return `None`.

Support: read the exact case aloud, enlarge the labelled picture, supply input cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

Extension: Add `print(result)` after the calls in a fresh script with no global result. Require a prediction and a reason before comparing. Expected result: `NameError` occurs because that local name is not defined in the caller. Reason: A returned value and the function’s internal name are different things.

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

Criterion · Not yet · With support · Independent

Criterion: Explanation

Not yet: Names an answer without a deciding fact

With support: Uses a hint to link fact and decision

Independent: Explains the deciding fact in own words

Criterion: Trace or test

Not yet: Cannot connect input and result

With support: Completes steps with prompts

Independent: Traces or tests a changed case accurately

Criterion: Limits and revision

Not yet: Claims more than evidence supports

With support: Names a limit after a prompt

Independent: States a defensible limit and useful next test

Project checkpoint: Python readiness portfolio. Keep runnable code, predictions, actual outputs, one repaired error and an explanation of a function on changed inputs.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and evidence.

8. Use feedback, labs and repair paths

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding evidence rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Ask students to predict before running code. Preserve errors and actual outputs; collect a changed test and an explanation. The branch-boundary and indexing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as evidence. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.